

SB-LandOwner-Sync Project — Technical Specification

Document type: Technical Specification | Project: SB-LandOwner-Sync | Version: 1.0.0 | Date: June 2026 | Platform: MuleSoft Anypoint Platform 4.6.0 | **Proposed Design** | Source: business-requirements.pdf

Overview

This integration synchronises **land-rights owner** data from **Shikun & Binui's Rainbow** system into **Salesforce**. The business need is to produce a "warning note" (הערת אזהרה), which requires an accurate, up-to-date list of who holds rights in a parcel of land and what share each holds.

Data originates from the Rainbow `registrations`, `owners` and `businessPartners` OData entities (database 262) and is written into the Salesforce custom object `LandOwner__c`. Per the business decision, records are **not** linked to a Salesforce `Account` and no `Project` is created — because Rainbow data is not always consistent and identifiers are sometimes missing, the information is written as a **flat record** directly into the "Land Owners" table via **upsert**. The integration runs as an unattended **nightly batch**.

1. Project Overview

Property	Value
Maven Group ID	<code>com.shikunbinui.integration</code> Proposed
Maven Artifact ID	<code>sb-landowner-sync</code> Proposed
Version	<code>1.0.0</code> Proposed
Packaging	<code>mule-application</code>
Mule Runtime	4.6.0 Proposed
Mule Maven Plugin	4.3.0 Proposed
Java Version	17 Proposed
HTTP Port	8081 (health/ops only — no public API) Proposed
Flow definition file(s)	<code>src/main/mule/landowner-sync.xml</code> , <code>src/main/mule/global-config.xml</code> , <code>src/main/mule/error-handling.xml</code>

2. Connector Dependencies

Connector	Group ID	Version	Purpose
<code>mule-http-connector</code>	<code>org.mule.connectors</code>	1.10.3 Proposed	Outbound HTTP/OData requests to Rainbow; inbound health listener
<code>mule-salesforce-connector</code>	<code>com.mulesoft.connectors</code>	11.1.0 Proposed	Salesforce <code>LandOwner__c</code> query, upsert and delete
<code>mule-sockets-connector</code>	<code>org.mule.connectors</code>	1.2.5 Proposed	Low-level socket support (HTTP connector runtime dependency)

3. Global Configurations

3.1 HTTP Request — `Rainbow_Analytics_HTTP_Request_config`

Property	Value
Base path host	<code>\${rainbow.analytics.host}</code> (e.g. <code>ziv-rainbow.net</code>)
Base path	<code>/api/analytics/odata</code>
Protocol	HTTPS
Port	443
Connection Timeout	<code>\${rainbow.timeout.ms}</code> (default 30000)
Authentication	HTTP Basic — <code>\${rainbow.username}</code> / <code>\${rainbow.password}</code> (secure)

3.2 HTTP Request — `Rainbow_Operation_HTTP_Request_config`

Property	Value
Base path host	<code>\${rainbow.operation.host}</code> (e.g. <code>ziv-rainbow.net</code>)
Base path	<code>/api/operation/odata</code>
Protocol	HTTPS
Port	443
Authentication	HTTP Basic — <code>\${rainbow.username}</code> / <code>\${rainbow.password}</code> (secure)

3.3 Salesforce — `Salesforce_Config`

Property	Value
Auth type	Basic (Username/Password + Security Token) <code>Proposed</code>
Username	<code>\${salesforce.username}</code>
Password	<code>\${salesforce.password}</code> (secure)
Security Token	<code>\${salesforce.securityToken}</code> (secure)
API version	<code>59.0</code> <code>Proposed</code>

3.4 HTTP Listener — `Health_HTTP_Listener_config`

Property	Value
Host	<code>0.0.0.0</code>
Port	<code>\${http.port}</code> (default 8081)

3.5 Configuration Properties — `config.yaml`

Property	Value
File	<code>src/main/resources/config.yaml</code> (+ <code>secure-config.yaml</code> via Secure Properties)

4. Flows

4.1 Flow: `landowner-sync-scheduler-flow`

doc: id: sched-main-0001

Trigger	Expression	Description
Scheduler	<code>\${sync.cron}</code> — default <code>0 0 2 * *</code> <code>?</code> (02:00 daily, Asia/Jerusalem)	Nightly batch that orchestrates the full Rainbow → Salesforce land-owner synchronisation

Processing Steps

- 1
- Log start — emit `START landowner-sync correlationId=#[correlationId]`
- 2
- Purge stale records — `flow-ref` to `delete-stale-landowners-subflow` (removes expired records, preserving manually-managed ones)

- 3 **Fetch current registrations** — `flow-ref` to `fetch-registrations-subflow`; result stored in `vars.registrations`
- 4 **Iterate registrations** — `<foreach>` over `vars.registrations`:
 - 4a. `flow-ref` to `build-landowners-for-registration-subflow` — fetch owners + business partner, build flat records
 - 4b. Accumulate built records into `vars.landownerBatch`
- 5 **Filter empties** — remove any registration that yielded zero owners (`vars.landownerBatch filter (not isEmpty($))`)
- 6 **Upsert to Salesforce** — `flow-ref` to `upsert-landowners-subflow` (batched upsert by external ID)
- 7 **Log summary** — emit `END landowner-sync upserted=#[vars.upsertCount] deleted=#[vars.deleteCount]`

4.2 Sub-flow: `delete-stale-landowners-subflow`

`doc:id: sub-delete-0002` Sub-flow called by 4.1 step 2.

Processing Steps

- 1 **Query expired records** — Salesforce `query`: `SELECT Id FROM LandOwner__c WHERE ToDate__c != null AND ToDate__c < TODAY AND Lawyer__c = null`
- 2 **Guard** — `<choice>`: if no rows, skip delete and set `vars.deleteCount = 0`
- 3 **Delete** — Salesforce `delete` on the returned `Id` list; capture count into `vars.deleteCount`

Rule: a record is "stale" only when its `ToDate__c` has passed. Records with an empty `ToDate__c` are never deleted. Records where `Lawyer__c` was filled manually by Shikun & Binui are excluded from deletion so manual work is preserved.

4.3 Sub-flow: `fetch-registrations-subflow`

`doc:id: sub-fetch-reg-0003` Sub-flow called by 4.1 step 3.

Processing Steps

- 1 **Request registrations** — HTTP GET on `Rainbow_Analytics_HTTP_Request_config`: `/registrations?$filter=isCurrentRegistration eq true`
- 2 **Page** — follow OData `@odata.nextLink` until exhausted (collect all pages)
- 3 **Map** — project each registration to `{ registrationKey, buildingCode, propertyKey, projectCode, rightType, plot }` into `vars.registrations`

4.4 Sub-flow: `build-landowners-for-registration-subflow`

`doc:id: sub-build-0004` Sub-flow called by 4.1 step 4a, once per registration.

Processing Steps

- 1 **Fetch owners** — HTTP GET `/owners?$filter=toDate eq null and registrationKey eq #[vars.reg.registrationKey]`
- 2 **Skip if no owners** — `<choice>`: if owners list is empty, return `[]` (registration filtered out per business rule)
- 3 **Enrich business partner** — for each owner, HTTP GET on `Rainbow_Operation_HTTP_Request_config : /businessPartners?$filter=id eq #[owner.businessPartnerId]` to obtain `idtype`, name, address, legal id
- 4 **Build flat records** — DataWeave transform combining registration + owner + business partner into `LandOwner__c` shape (see Section 9.1)
- 5 **Resolve Building lookup** — `<choice>`: if `buildingCode` present, set `Building__r.BuildingCode__c` for the relationship; otherwise leave building null (record linked at project level only)

4.5 Sub-flow: `upsert-landowners-subflow`

`doc.id: sub-upsert-0005` Sub-flow called by 4.1 step 6.

Processing Steps

- 1 **Chunk** — split `vars.landownerBatch` into chunks of `${salesforce.batchSize}` (default 200)
- 2 **Upsert** — Salesforce `upsert` on `LandOwner__c` using external ID field `External_Key__c`
- 3 **Collect results** — accumulate success/failure counts; log any item-level failures with their `External_Key__c`

Response / Outcome

Outcome	Channel	Detail
Success	Log	<code>END landowner-sync upserted=N deleted=M</code>
Partial failure	Log (WARN)	Per-record <code>External_Key__c</code> + Salesforce error message
Fatal failure	Global error handler	Logged at ERROR; run marked failed (see Section 6)

5. Configuration Properties

Property Key	Default / Placeholder	Description
<code>http.port</code>	<code>8081</code>	Local health-listener port

<code>sync.cron</code>	<code>0 0 2 * * ?</code>	Nightly schedule (Asia/Jerusalem)
<code>rainbow.analytics.host</code>	<code>ziv-</code> <code>rainbow.net</code>	Rainbow Analytics OData host
<code>rainbow.operation.host</code>	<code>ziv-</code> <code>rainbow.net</code>	Rainbow Operation OData host
<code>rainbow.username</code>	<code>CHANGE_ME</code>	Rainbow service account username (<i>replace before running</i>)
<code>rainbow.password</code>	<code>CHANGE_ME</code>	Rainbow service account password (<i>secure — replace before running</i>)
<code>rainbow.timeout.ms</code>	<code>30000</code>	HTTP connection/response timeout
<code>salesforce.username</code>	<code>CHANGE_ME</code>	Salesforce integration user (<i>replace before running</i>)
<code>salesforce.password</code>	<code>CHANGE_ME</code>	Salesforce password (<i>secure — replace before running</i>)
<code>salesforce.securityToken</code>	<code>CHANGE_ME</code>	Salesforce security token (<i>secure — replace before running</i>)
<code>salesforce.batchSize</code>	<code>200</code>	Upsert chunk size

6. Error Handling Strategy

Error Type	Scope	Behaviour
<code>HTTP:CONNECTIVITY</code> / <code>HTTP:TIMEOUT</code> (Rainbow)	On Error Continue (per registration)	Retry up to 3× with backoff; on final failure, log the failing <code>registrationKey</code> and continue with the next registration (no abort)
<code>SALESFORCE:CONNECTIVITY</code>	On Error Propagate (global)	Abort run, log at ERROR; next nightly run retries the full set (idempotent upsert)
<code>SALESFORCE:INVALID_FIELD</code> / item failures	On Error Continue (upsert)	Log per-record error with <code>External_Key__c</code> ; remaining records still committed
<code>ANY</code> (uncaught)	Global default error handler	Log full error + correlationId at ERROR level

7. Logging Convention

Event	Level	Message Pattern
Run start	INFO	<code>START landowner-sync correlationId={id}</code>
Purge complete	INFO	<code>Deleted {n} stale LandOwner__c records</code>
Registrations fetched	INFO	<code>Fetches {n} current registrations</code>
Registration skipped	DEBUG	<code>Skipping registrationKey={k} – no owners</code>
Rainbow call failed	WARN	<code>Rainbow fetch failed registrationKey={k}: {msg}</code>
Upsert item failed	WARN	<code>Upsert failed externalKey={ek}: {sfError}</code>
Run end	INFO	<code>END landowner-sync upserted={n} deleted={m} skipped={s}</code>

Log file `sb-landowner-sync.log`, rolling at 10 MB with 10 archived files retained (proposed log4j2 policy).

8. External API Reference

8.1 Rainbow — Analytics OData

Attribute	Value
Base URL	<code>https://ziv-rainbow.net/api/analytics/odata</code>
Method	GET
Endpoints	<code>/registrations?\$filter=isCurrentRegistration eq true</code> <code>/owners?\$filter=toDate eq null and registrationKey eq {key}</code>
Response fields used	registrations: <code>registrationKey, buildingCode, propertyKey, righttype, project_code, plot</code> ; owners: <code>businessPartnerId, ownerspercent, fromDate, toDate</code>
Authentication	HTTP Basic <code>\${rainbow.username}</code> / <code>\${rainbow.password}</code>

8.2 Rainbow — Operation OData

Attribute	Value
Base URL	<code>https://ziv-rainbow.net/api/operation/odata</code>
Method	GET
Endpoint	<code>/businessPartners?\$filter=id eq {id}</code>

Response fields used	<code>buisnesspartnername</code> , <code>buisnesspartnerlegalidnumber</code> , <code>buisnesspartneradress</code> , <code>idtype</code>
Authentication	HTTP Basic <code>\${rainbow.username}</code> / <code>\${rainbow.password}</code>

8.3 Salesforce

Attribute	Value
Object	<code>LandOwner__c</code>
Operations	<code>query</code> (stale lookup), <code>delete</code> , <code>upsert</code> (external ID <code>External_Key__c</code>)
Authentication	Username/Password + Security Token

9. Data Transformation Detail

9.1 Build flat `LandOwner__c` record — combines a Rainbow registration, its owner, and the owner's business partner into a single flat Salesforce record. Field mapping is taken directly from the requirements document. `External_Key__c` is the Rainbow **owner record id** (confirmed), used for idempotent upsert. `Project__c` is written as an external-id lookup relationship (`Project__r`).

DataWeave:

```
%dw 2.0
output application/java
// inputs: vars.reg (registration), owner, bp (businessPartner)
---
{
  External_Key__c: owner.id as String,           // Rainbow owner record id (upsert key)
  Name: (bp.buisnesspartnername default "")[0 to 79], // required, max 80
  Idtype__c: bp.idtype,                          // ת.ז. / הפ.ח
  IdNumber__c: owner.buisnesspartnerlegalidnumber default bp.buisnesspartnerlegalidnumber,
  Address__c: bp.buisnesspartneradress,
  Essence__c: vars.reg.righttype,                // מהות הזכות
  OwnershipPercent__c: owner.ownerspercent,
  // Project__c is a lookup – related by external code via Project__r
  Project__r: { Project_Code__c: vars.reg.project_code }, // mandatory
  BuildingCode__c: vars.reg.buildingCode,
  Plot__c: vars.reg.plot,
  FromDate__c: owner.fromdate as Date,
  ToDate__c: owner.toDate as Date default null
}
```


9.2 Conditional Building lookup — when a `buildingCode` exists the record is related to a `Building__c` via external-id relationship; otherwise the building reference is omitted and the record stands at project level only.

DataWeave:

```
%dw 2.0
output application/java
---
payload map ((rec) ->
  if (rec.BuildingCode__c != null)
    rec ++ { Building__r: { BuildingCode__c: rec.BuildingCode__c } }
  else
    rec - "BuildingCode__c"
)
```

10. Build & Run

```
# Build
mvn clean package

# Build without tests
mvn clean package -DskipTests

# Deploy to local Mule runtime
mvn clean package -DmuleDeploy
```

11. Sample Request & Response

The integration is scheduler-driven (no inbound API). The samples below show the outbound Rainbow OData calls and the resulting Salesforce upsert payload.

```
# 1. Current registrations
GET https://ziv-rainbow.net/api/analytics/odata/registrations?$filter=isCurrentRegistration e

# 2. Owners for a registration
GET https://ziv-rainbow.net/api/analytics/odata/owners?$filter=toDate eq null and registrati

# 3. Business partner enrichment
GET https://ziv-rainbow.net/api/operation/odata/businessPartners?$filter=id eq 11
```

```
# Resulting Salesforce upsert record (LandOwner__c)
{
  "External_Key__c": "55021",
```

```
"Name": "שיכון ובינוי גדלן בע"מ",
"Idtype__c": "ה.פ.",
"IdNumber__c": "520021171",
"Address__c": "הירדן 1א",
"Essence__c": "הכירה",
"OwnershipPercent__c": 100,
"Project__r": { "Project_Code__c": "PRJ-1024" },
"BuildingCode__c": "7976",
"Building__r": { "BuildingCode__c": "7976" },
"FromDate__c": "2024-01-01",
"ToDate__c": null
}
```

12. Design Assumptions & Open Questions

The four high-impact items below were **confirmed by the client** and are reflected in the design above. The remaining open questions are operational and do not block development.

Confirmed Decisions

Item	Confirmed decision
Stale-record purge driver	Add <code>ToDate__c</code> on <code>LandOwner__c</code> mapped from <code>owners.toDate</code> ; delete where <code>ToDate__c < TODAY</code> AND <code>Lawyer__c = null</code> (empty <code>ToDate__c</code> and lawyer-managed records preserved) Confirmed
Upsert external key	<code>External_Key__c</code> = Rainbow owner record id (<code>owner.id</code>) Confirmed
Rainbow OData authentication	HTTP Basic — service username / password (secure properties) Confirmed
<code>Project__c</code> population	Lookup relationship written by external code via <code>Project__r.Project_Code__c</code> = <code>project_code</code> Confirmed

Remaining Design Assumptions

Assumption	Rationale	Impact if wrong
Mule runtime 4.6.0, Java 17, batch size 200, 3 retries	Current supported defaults; no NFRs given	May need tuning for very high registration volumes
Building is matched by <code>BuildingCode__c</code> as an external-id relationship on <code>Building__c</code>	Spec says "link to building via lookup if building exists" but not the match key	If Building lookup uses a different key, the relationship insert fails

Schedule fixed at 02:00 Asia/Jerusalem	Spec says "once nightly" without a time	May clash with Rainbow maintenance windows
--	---	--

Open Questions (non-blocking)

Question	Why it matters	Suggested default
Should the run send a completion / failure notification (email, Slack, ops dashboard)?	Unattended nightly batch needs operational visibility	Email on fatal failure
Expected volume of current registrations per night?	Drives paging, batch size and SLA	Assume < 50k records
Confirm the Building lookup match key on <code>Building__c</code>	Determines the <code>Building__r</code> external-id field name	<code>BuildingCode__c</code>